

Document Number: P3248R0.

Date: 2024-04-17.

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>.

Authors: Gonzalo Brito Gadeschi.

Audience: SG1, SG22, EWG, LEWG.

Require `[u]intptr_t`

- Require `[u]intptr_t`
 - Motivation
 - Implementation Experience
 - Design
 - Usage Guideline
 - Wording changes

Motivation

Proposals like P2835 (<https://wg21.link/p2835>) and P3125 (<https://wg21.link/P3125>) require an integer type capable of holding a pointer value in their APIs. However, `[u]intptr_t` being *optional* forces sub-optimal design choices such as making APIs optional or introducing workarounds.

The potential absence of `[u]intptr_t` compromises the portability of high-level software and attempts to address this introduce software engineering overheads and potential portability bugs, as seen in libvlc PR#1519 (https://code.videolan.org/videolan/vlc/-/merge_requests/1519).

This proposal advocates for requiring `[u]intptr_t` in C++ to ensure that all C++ code can rely on integer types capable of holding a pointer value.

Implementation Experience

A survey found almost unanimous support for `[u]intptr_t` in C++:

- C++ Standard Library implementations that assume `[u]intptr_t` are available: `libstdc++`, `libc++`, and Microsoft STL.
- C++ Compilers supporting `[u]intptr_t` on all targets, including those with non-standard pointer sizes: GCC, Clang, MSVC.
- C++ Platform ABIs specify the size and alignment of pointers and the calling convention of Integer types, fixing the ABI of `[u]intptr_t`. *Extended integer types* avoid breaking

the ABI of `intmax_t` when introducing a wider `[u]intptr_t` (this used to be a problem, see N2889 (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2889.htm>)).

Platforms that do not provide `[u]intptr_t` types today would be impacted:

- **Implementability:** integer widths without native HW support can be implemented using SW emulation sequences. LLVM and GCC support arbitrary-sized integers (<https://llvm.org/docs/LangRef.html#integer-type>), which is required for C23 `_BitInt` support (321-bit demo (<https://godbolt.org/z/h4Mzjrxs6>)). While LLVM makes no guarantees about the ABI of `iX` types, Clang documents their ABI as unstable (see Clang's C23 `_BitInt` documentation (<https://clang.llvm.org/docs/LanguageExtensions.html#id49>); D115169 (<https://reviews.llvm.org/D115169>) proposes stabilizing these), and GCC avoids unstable ABIs by only exposing this functionality on targets with stable ABIs (x86 and aarch64). An ABI for `[u]intptr_t` may be required. Other toolchains may also need to implement code-generation for new integer widths.
- **Performance:** SW-emulation of integer widths on platforms lacking hardware support are zero-cost abstractions that perform as efficiently as the architects of those systems intended.

Therefore, we conclude that:

- **C++ platform support:** requiring `[u]intptr_t` does not regress C++ platform support.
- **Implementation effort:** most platforms provide them (unanimous support above), platforms using toolchains with arbitrary-sized integers may need to specify an ABI for them, other platforms may need to also implement code generation.

Design

Design alternatives:

1. C++ requires `[u]intptr_t`.
2. C++ adds new integer types - different from `[u]intptr_t` - capable of holding a pointer value.
3. Do nothing.

This proposal advocates for Option 1, i.e., for C++ to require `[u]intptr_t`, because:

- **Pre-existing code:** All implementations surveyed provide these on all platforms. This has led to a large corpus of pre-existing code using `[u]intptr_t`. Requiring `[u]intptr_t` makes this code portable to all platforms C++ supports. Inventing new C++ types would make this code non-idiomatic and cause significant churn on all ecosystems for little added value.

- **Compatibility with C:** By requiring these types in C++ with the same semantics as C we ensure C++ remains compatible with C. When C provides these types, C++ code using these in a C-compatible remains source and ABI compatible with C. Adding new C++ types that are not available in C would reduce C++'s compatibility with C.
- **ABI:** Platforms whose ABI specifies `intmax_t` to be smaller than the platform's pointer size are allowed to provide wider `[u]intptr_t` integer types since C23 and C++23 due to extended integer type support.
- **Cost:** There is a cost to doing nothing. Significant time was spent on `atomic_ref::address` to find a sub-optimal solution when the right solution everyone agrees on is `uintptr_t`.

Usage Guideline

`[u]intptr_t` is well suited for C++ language or C++ Standard Library APIs that need an integer type capable of holding a *pointer value*, i.e., an integer type with a lossless conversion from/to pointer.

Some features or APIs may only need an integer type capable of holding a *pointer address*. C and C++ do not provide an integer type suited for this use case, but some standard libraries do, e.g., glibc's `<stdint.h>` header `ptraddr_t` on Arm CHERI.

Wording changes

Modify `[cstdint.syn]` (<https://eel.is/c++draft/cstdint.syn#1>):

1. The header `<cstdint>` supplies integer types having specified widths, and macros that specify limits of integer types.

```

// all freestanding
namespace std {
    using int8_t      = signed integer type;    // optional
    using int16_t     = signed integer type;    // optional
    using int32_t     = signed integer type;    // optional
    using int64_t     = signed integer type;    // optional
    using intN_t      = see below;              // optional

    using int_fast8_t = signed integer type;
    using int_fast16_t = signed integer type;
    using int_fast32_t = signed integer type;
    using int_fast64_t = signed integer type;
    using int_fastN_t = see below;              // optional

    using int_least8_t = signed integer type;
    using int_least16_t = signed integer type;
    using int_least32_t = signed integer type;
    using int_least64_t = signed integer type;
    using int_leastN_t = see below;             // optional

    using intmax_t     = signed integer type;
    using intptr_t     = signed integer type;    // optional

    using uint8_t      = unsigned integer type; // optional
    using uint16_t     = unsigned integer type; // optional
    using uint32_t     = unsigned integer type; // optional
    using uint64_t     = unsigned integer type; // optional
    using uintN_t      = see below;             // optional

    using uint_fast8_t = unsigned integer type;
    using uint_fast16_t = unsigned integer type;
    using uint_fast32_t = unsigned integer type;
    using uint_fast64_t = unsigned integer type;
    using uint_fastN_t = see below;             // optional

    using uint_least8_t = unsigned integer type;
    using uint_least16_t = unsigned integer type;
    using uint_least32_t = unsigned integer type;
    using uint_least64_t = unsigned integer type;
    using uint_leastN_t = see below;            // optional

    using uintmax_t    = unsigned integer type;
    using uintptr_t    = unsigned integer type; // optional
}

#define INTN_MIN      see below
#define INTN_MAX      see below
#define UINTN_MAX     see below

```

```

#define INT_FASTN_MIN    see below
#define INT_FASTN_MAX    see below
#define UINT_FASTN_MAX   see below

#define INT_LEASTN_MIN   see below
#define INT_LEASTN_MAX   see below
#define UINT_LEASTN_MAX   see below

#define INTMAX_MIN       see below
#define INTMAX_MAX       see below
#define UINTMAX_MAX      see below

#define INTPTR_MIN       see below           // optional
#define INTPTR_MAX       see below           // optional
#define UINTPTR_MAX      see below           // optional

#define PTRDIFF_MIN      see below
#define PTRDIFF_MAX      see below
#define SIZE_MAX         see below

#define SIG_ATOMIC_MIN   see below
#define SIG_ATOMIC_MAX   see below

#define WCHAR_MIN        see below
#define WCHAR_MAX        see below

#define WINT_MIN         see below
#define WINT_MAX         see below

#define INTN_C(value)    see below
#define UINTN_C(value)   see below
#define INTMAX_C(value)  see below
#define UINTMAX_C(value) see below

```

2. The header defines all types and macros the same as the C standard library header `<stdint.h>` except that the types `intptr_t` and `uintptr_t` and the macros `INTPTR_MIN`, `INTPTR_MAX`, and `UINTPTR_MAX` are always defined and are not optional. See also: ISO/IEC 9899:2018, 7.20.

3. All types that use the placeholder N are optional when N is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for N = 8, 16, 32, and 64 are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding typedef-names. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding

typedef-name.

[Note 1: The macros INTN_C and UINTN_C correspond to the typedef-names `int_leastN_t` and `uint_leastN_t`, respectively. — end note]